MONAGENG TUTORING

Tutoring Booking & Student Management System

System requirements, analysis, database design, business rules, security, testing, and installation guide.

Course: HPXS302-1 — Praxis S3B
Assessment: Summative Assessment 1, July–December 2026
Student: Kgalalelo Monageng
Student number: 30HA2400106
Implementation: PHP 8, MariaDB/MySQL, HTML5, CSS, JavaScript

Prepared 28 October 2026. Test payments are simulated and no card details are collected.

1. System overview

Monageng Tutoring supports students taking Mathematics and Data Science tutoring programmes. Students register, submit a programme payment for administrator verification, and book weekday sessions after enrolment activation. Tutors manage availability and respond to their assigned sessions. Administrators maintain records and verify payments.

The web application is implemented with server-rendered PHP and MariaDB. HTML5 form constraints and JavaScript improve usability; PHP repeats security-critical validation. Session role checks protect student, tutor, and administrator pages.

2. Requirements

Functional requirements

1. **Registration and authentication:** students self-register. Students, tutors, and administrators authenticate by email and password; successful login opens a role-specific dashboard.
2. **Programme payment and enrolment:** students select a Mathematics or Data Science programme. The application records a pending payment and pending enrolment; an administrator verifies the payment, after which the enrolment becomes active.
3. **Session booking and tutor allocation:** students with an active enrolment request a Monday–Friday session. The system matches programme subject to tutor expertise and an available time, assigns one tutor, and records the request.
4. **Tutor schedule and sessions:** tutors maintain their own available time slots, view assigned sessions, confirm requests, complete sessions, or cancel/decline sessions.
5. **Administration and communication:** administrators manage students, tutors, programmes, bookings, availability, and payment verification. Booking events create in-app notifications for the student, assigned tutor, and administrator.

Non-functional requirements

1. **Security:** protect credentials with password hashing, role checks, prepared SQL, CSRF tokens on sensitive changes, validated input, and escaped HTML output.
2. **Reliability:** use database transactions for payment/enrolment, booking/notification, and session-status/notification operations.
3. **Usability:** responsive pages, labelled forms, HTML5 input types and required constraints, clear validation messages, and role-specific navigation.
4. **Performance:** indexed relationship and lookup columns, bounded administrative lists, and targeted parameterized queries.
5. **Maintainability and growth:** separate PHP, HTML, CSS, JavaScript, and SQL assets; normalized data; foreign keys and indexes support consistent extension.

3. Process and business rules

- Only an authenticated student with an active enrolment in the selected programme can request a session for it.
- Session dates must be valid weekdays and not earlier than the current date.
- A tutor must be active, have expertise matching the programme, and have an available slot that is not already booked by a requested or confirmed session.
- A booking is initially *Requested*. The assigned tutor or administrator can confirm it; a confirmed session can be completed or cancelled. Students can cancel their own requested or confirmed sessions.
- State-changing booking actions are checked against the user's role and ownership, then recorded with participant notifications in one database transaction.
- Payment is a demonstration record, not a payment-gateway transaction. Enrolment remains pending until administrator verification.
- Notifications are stored in the application and shown to the relevant signed-in user. Email/SMS delivery is not configured.

4. Database design and normalization

The fresh schema is in `database/schema.sql`. Primary keys identify records; foreign keys preserve relationships. All tables use InnoDB and UTF-8. Currency is stored as `DECIMAL(10,2)` and displayed in South African rand.

Entity	Purpose and key relationships
students	Student identity, unique email, contact details, hashed password.
tutors	Tutor identity, unique email, hashed password, subject expertise and status.
administrators	Administrator identity, unique email and hashed password.
programmes	Programme name, description and fee.
payments	Student and programme payment record, amount, date and verification status.
enrolments	One enrolment per payment, date and activation status. Student/programme are obtained through the referenced payment.
bookings	Student, tutor, programme, date/time and lifecycle status.
tutor_availability	Tutor's unique date/time slot and availability status.

Entity	Purpose and key relationships
notifications	Booking event, recipient role/identifier, message and timestamp.

Normalization summary

- **UNF:** a single student/programme/payment/tutor/session sheet repeats student and programme values for each session.
- **1NF:** each entity has a primary key and atomic attributes; repeated session rows are stored as individual bookings.
- **2NF:** tables use single-column primary keys, and descriptive attributes depend on their whole key.
- **3NF:** student, tutor, administrator, programme, payment, enrolment, booking, availability, and notification facts are separated. Enrolments reference a unique payment; student/programme values are derived through that payment, avoiding duplicate dependencies.

5. Application and access control

Role	Permitted activities
Student	Register, log in, submit programme payment, book an active programme's available weekday slot, inspect/cancel own sessions, view own notifications.
Tutor	Log in, manage own availability, view assigned sessions, confirm/decline requests, complete/cancel confirmed sessions, view own notifications.
Administrator	Manage students, tutors, programmes, and bookings; maintain any tutor's availability; verify payments and inspect system notifications.

Passwords are created with PHP `password_hash()` and checked with `password_verify()`. Prepared statements are used for user-provided SQL values. Displayed database content is escaped with `htmlspecialchars()`. Session cookies are HttpOnly and SameSite=Lax, with Secure enabled for HTTPS.

6. Testing and evidence

Check	Result
PHP syntax checks across application PHP files	Passed during implementation.

Check	Result
Fresh schema import on MariaDB 10.11	Passed; tables, indexes, and foreign keys created successfully.
Migration and demo seed re-import	Passed on an isolated legacy-shaped schema; seed can be imported twice without duplicate demo records.
Payment verification and automatic enrolment activation	Passed on the local XAMPP app with the synthetic pending Data Science account.
Active student booking, tutor assignment, confirmation, and completion	Passed end to end; subject match, role access, and participant notification records verified.
PHP and JavaScript syntax	All application PHP files and `js/main.js` passed their syntax checks.
Live card charging, email/SMS delivery, and production load	Out of scope for this educational demonstration; no gateway or message provider is configured.

Browser checks confirmed that programme selection populates matching dates and the selected date populates available times with tutor names. The administrator booking list presents student and tutor names and email addresses, and the programme name, while retaining booking IDs for editing. Screenshots of registration, login, student payment and booking, student dashboard and notifications, tutor dashboard and availability, and administrator dashboard and booking management are included in `docs/screenshots/` and use the fictional assignment sample data. Follow `docs/README.md` to review the sample accounts and repeat the workflow in the assessment environment.

7. Setup and run

1. Start Apache and MySQL in XAMPP.
2. For a fresh install, import `database/schema.sql` and then `database/sample_data.sql` in phpMyAdmin.
3. For the supplied existing local schema, make a database backup, then run `database/migrate_submission_features.sql` once and import the sample data.
4. Open `http://localhost/TutoringManagementSystem/`.
5. Use the assignment sample account credentials listed in `docs/README.md`. The shared sample password is `Monageng@1`.

Replace demonstration credentials and local database credentials before any non-local deployment. Do not redistribute the excluded legacy SQL dump; it contains personal information and plaintext passwords.